

Forecasting Online Product Sales

Improvements, Extensions & Comparative Analysis

NeuralForecast Models with Cluster-Level Statistical Analysis

GitHub: <https://github.com/bhuvighosh3/Forecasting-project/tree/Retail>

Authors: Anamika Mishra(akm2259), Afreen Sorathiya(as7852), Bhuvi Ghosh(bpg2131)

UC Irvine Online Retail II Dataset | May 2026

NHITS · NBEATS · PatchTST · TiDE · TimesNet · Prophet

Table of Contents

1. Abstract	3
2. Introduction	3
2.1 Contributions Over Prior Team	3
3. Background Research and Related Work	4
3.1 Prior Team Model Framework	4
3.2 New Models Introduced in This Study	4
3.3 LLM-Based Clustering — Prior Team Innovation	5
4. Data	6
4.1 Dataset Overview	6
4.2 Preprocessing Pipeline Comparison	6
4.3 Feature Engineering	7
5. Clustering — Different Basis, Not a Re-Run	8
5.1 Why We Chose Time-Series Similarity	8
5.2 Our Clustering Results	8
5.3 Methodology Difference vs Prior Team	9
6. Statistical Analysis per Cluster	10
6.1 Per-Cluster Statistical Profile	10
6.2 Kruskal-Wallis Significance Tests	11
7. Outlier Treatment	11
8. Models	12
8.1 Cluster-to-Model Assignment	12
8.2 Training Configuration & Hyperparameter Tuning	13
9. Results and Analysis	14
9.1 Prior Team Results — Real Notebook Outputs	14
9.2 Our Final SMAPE Results per Cluster	16
9.3 SMAPE Distribution Across Time Windows (Figure)	17
9.4 MAPE-Based Interpretation of Performance Tiers	18
9.5 Methodology Comparison with Prior Team	19
10. Discussion	20
11. Conclusion	22
Appendix A Prior Team Report Summary	23
References	25

1. Abstract

This report presents a systematic improvement to the prior team's online retail demand forecasting study (Ta, Ma, do Valle, Mah — Columbia University, May 2025). The prior team benchmarked seven models (Prophet, RNN/LSTM, TFT, DeepAR, GLM, Decision Tree, XGBoost) on the UCI Online Retail II dataset, introducing a novel LLM-based clustering pipeline using Gemini 2.5 description embeddings. Their best reported sMAPE results — extracted directly from their notebooks — were Prophet on total aggregated sales at 31.83%, TFT at a per-product mean of 39.84% at Horizon 1, XGBoost at 35.14% on validation, RNN/LSTM at 40.49% on test, GLM at 51.21% mean at Horizon 1, and DeepAR at 92.51% baseline (89.81% with KMeans clustering).

Our study introduces four targeted improvements: (1) cluster-level statistical analysis (CV%, ADF/KPSS stationarity, kurtosis, seasonal decomposition) drives data-aware model selection; (2) Z-score-based outlier replacement at the StockCode level handles heavy-tailed clusters (replacing 2,127 outliers in Cluster 0 alone); (3) modern NeuralForecast architectures (NHITS, NBEATS, PatchTST, TiDE, TimesNet) replace the prior team's LSTM backbone; and (4) multi-metric evaluation (SMAPE, MAE, RMSE, MASE) versus a cyclical naive baseline confirms genuine forecasting skill. We also adopt rolling-origin cross-validation across four time windows (Late 2010, Early 2011, Mid 2011, Late 2011) using 30-day sub-windows.

Our final results show that Prophet_Tuned with significant features (calendar + lag + rolling) wins 4 of 12 clusters (Clusters 7, 9, 10, 11) with SMAPE between 53.56% and 86.41%; NHITS_SigFeatures wins 3 clusters (1, 4, 8) with SMAPE between 82.98% and 99.40%; NBEATS wins Cluster 0 (the largest, 4,008 products) with the best overall SMAPE of 49.65%; NHITS without features wins Cluster 3 at 89.76%; PatchTST wins the two highest-CV% sparse clusters (2, 6) with SMAPE 138.18–189.04%; and Prophet_Tuned (no features) wins Cluster 5 at 183.62%. Cluster 0 (NBEATS at 49.65%), Cluster 10 (Prophet_Tuned_SigFeatures at 53.56%), and Cluster 7 (Prophet_Tuned_SigFeatures at 57.75%) constitute the high-skill tier; Clusters 1, 3, 4, 8, 9, 11 form the mid-tier (82–99% SMAPE); and Clusters 2, 5, 6 represent the unforecastable tier (>138% SMAPE) — single-product clusters with extreme volatility (CV% 424–714%) where no model can extract meaningful forward signal from largely zero-valued series. Our overall mean SMAPE across all 12 clusters is 100.59%.

2. Introduction

Accurate product-level demand forecasting is a core operational challenge in online retail. Forecasts inform inventory purchasing, warehouse staffing, promotional planning, and customer delivery promises. For a UK-based gift retailer selling thousands of products — as in this dataset — the challenge is compounded by highly seasonal demand (Christmas, Black Friday, Valentine's Day), extreme heterogeneity across products, and significant data sparsity (only two years of daily history with many products having zero sales on most days).

The prior team built a comprehensive benchmark, evaluating seven model architectures from simple GLM through state-of-the-art TFT and DeepAR. Their most significant innovation was introducing LLM-based clustering via Gemini 2.5 embeddings of product descriptions — grouping semantically similar products for joint training. Their results showed that deep learning models (TFT, DeepAR) were competitive even without clustering, while simpler models (GLM, Decision Tree) benefited more from explicit feature engineering. Prophet achieved the best aggregate-level result at 31.83% SMAPE, but this reflects variance cancellation from summing 142 product series rather than per-product accuracy.

Our study builds directly on this foundation, targeting four specific gaps: the absence of statistical analysis per cluster to guide model assignment; the lack of formal outlier treatment; the use of SMAPE as the sole evaluation metric (without comparison to a naive baseline); and the use of an older generation of neural architectures. We retain the same UCI Online Retail II dataset, daily granularity, and 60-day evaluation horizon, but restructure the pipeline around the NeuralForecast library with cluster-aware configuration and rolling-origin evaluation.

2.1 Contributions Over Prior Team

- Statistics-driven model selection: CV%, ADF/KPSS stationarity, kurtosis, and seasonal decomposition per cluster directly determine which model architecture is assigned. This is a principled replacement for the prior team's qualitative capability matrix.
- Richer per-StockCode feature engineering: lag_1, lag_7, lag_14 days plus rolling_mean_7, rolling_mean_14, and rolling_std_7 computed at individual product level before cluster aggregation — capturing per-product momentum rather than diluted cluster-level signals.
- Expanded UK calendar: is_saturday, is_sunday, is_black_friday, is_cyber_monday, is_christmas_period, is_year_end, and is_january added beyond the prior team's set.
- Formal outlier treatment: Z-score replacement ($|Z| > 3 \rightarrow$ cluster median) for clusters with kurtosis > 5 ; Cluster 8 (only non-stationary cluster, ADF $p=0.6178$) handled via differencing rather than replacement.
- Modern neural architectures: NHITS, NBEATS, PatchTST, TiDE, TimesNet via NeuralForecast API — architectures specifically designed for multi-horizon forecasting with more efficient training than RNN/LSTM.
- Rolling-origin evaluation: SMAPE measured across four time windows (Late 2010, Early 2011, Mid 2011, Late 2011) with 30-day sub-windows — providing a robust performance estimate rather than a single 60-day snapshot.
- Multi-metric evaluation: SMAPE + MAE + RMSE + MASE versus a cyclical weekly naive baseline. $MASE < 1.0$ confirms genuine forecasting skill beyond a trivial predictor.

3. Background Research and Related Work

3.1 Prior Team Model Framework

The prior team structured model selection around five criteria: multi-series capability, static feature support, cold-start handling, interpretability, and clustering requirements. Table 1 extends their framework with the new models we introduce.

Model	Multi-Series	Static Feats	Cold Start	Interpretable	Exog Support
Prophet (Prior)	No	Limited	No	High	Time-varying regressors
RNN/LSTM (Prior)	Partial	Yes	No	Low	Yes — embedded
TFT (Prior)	Yes	Yes	Yes	Medium	Future + static + historical
DeepAR (Prior)	Yes	Yes	Yes	Low	Categorical + static
GLM (Prior)	No	Manual	No	High	Manual feature engineering
Decision Tree (Prior)	No	Yes	No	High	All features as inputs
XGBoost (Prior)	No	Yes	No	Medium	All features as inputs
NHITS (Ours)	Yes	Yes	Yes	Low	Future + Historical exog
NBEATS (Ours)	Yes	No	No	Medium	None — pure history
PatchTST (Ours)	Yes	No	No	Low	None — patch self-attention
TiDE (Ours)	Yes	Yes	Yes	Low	Future + Historical exog
TimesNet (Ours)	Yes	Yes	Yes	Low	Future exog only

Table 1: Expanded model capability comparison. Prior team models in upper rows; our new models in lower bold rows.

3.2 New Models Introduced in This Study

3.2.1 NHITS — Neural Hierarchical Interpolation

NHITS decomposes the forecast hierarchically across multiple temporal scales using multi-rate input pooling and interpolation. Three stacks — coarse trend, medium seasonality, fine residual — each receive a differently sub-sampled version of the input, enabling the model to capture both slow trend changes and fast weekly cycles simultaneously. NHITS supports both future-known covariates (calendar features) and historical exogenous inputs (lag features). Configured with `n_pool_kernel_size=[2,2,1]`, `n_freq_downsample=[4,2,1]`, 512-unit MLP stacks.

3.2.2 NBEATS — Neural Basis Expansion Analysis

NBEATS uses fully connected residual blocks with backward and forward projections; the final forecast is the sum of forward projections across blocks. Interpretable variants use polynomial and harmonic basis functions to decompose predictions into explicit trend and seasonality components. NBEATS accepts no exogenous inputs — it relies entirely on the historical target series, making it

an extremely strong baseline for clusters where the sales pattern is the primary signal. Configured with `n_harmonics=2`, `n_polynomials=2`, 3 blocks per stack.

3.2.3 PatchTST — Patch Time Series Transformer

PatchTST divides the input series into fixed-length overlapping patches (`patch_len=16`, `stride=8`) and applies multi-head self-attention (`n_heads=4`) across patches rather than individual time steps. This dramatically reduces attention complexity while preserving long-range dependency capture. PatchTST accepts no exogenous variables, making it ideal for extremely sparse high-CV clusters where lag features computed on largely-zero series provide no forward signal.

3.2.4 TiDE — Time-series Dense Encoder

TiDE introduces a dense encoder-decoder architecture with a residual connection from encoder output to decoder input. Both future-known covariates and historical exogenous inputs project into the same latent space as the historical series. TiDE is designed for long-horizon forecasting and excels when calendar features and lag values jointly determine future demand.

3.2.5 TimesNet

TimesNet transforms the 1D time series into a 2D representation by detecting the dominant period and reshaping into a $(T/p) \times p$ matrix. 2D convolutions then capture patterns along the time axis (inter-period changes) and within each period (intra-period structure). Accepts future exogenous inputs only.

3.3 LLM-Based Clustering — Prior Team Innovation

The prior team's most novel contribution is their LLM-based clustering pipeline. They extracted text embeddings from product descriptions using Gemini 2.5, then reduced dimensionality with UMAP and PCA before applying clustering algorithms. They compared the same prompt across LLaMA 2, DeepSeek, GPT-4.1, and Claude 3.7 Sonnet. Their DBSCAN on UMAP-reduced Gemini 2.5 embeddings achieved a Silhouette Score of 0.88 with 23 clusters — the highest result in their study.

This semantic approach has a critical operational advantage for cold-start scenarios: a new product that has never been sold can be assigned to a cluster based on its description embedding alone, enabling forecasting before any sales data exists. Our approach (described in Section 5) takes a different basis — clustering by historical sales pattern similarity. We did not re-run the prior team's clustering; we adopted a different methodology that answers a different question. The two approaches are complementary, and a production system would integrate both.

4. Data

4.1 Dataset Overview

Both this study and the prior team use the UCI Online Retail II dataset, which contains all transactions from December 1, 2009 to December 9, 2011 for a UK-based online-only gift retailer. The dataset records each transaction with: Invoice ID (cancelled orders prefixed 'C'), StockCode (unique product identifier), Description (human-readable name), Quantity, InvoiceDate (minute-level), Price per unit (GBP), CustomerID, and Country.

Our notebook combines two original Excel sheets into a single dataframe of 1,067,371 transaction rows (Cell 7 output: 'Combined shape: (1067371, 8)'). After UK-only filtering and aggregation to daily level, the panel covers 739 days. The prior team produced 142 products × 739 days. Our StockCode-based pipeline retains 4,442 unique StockCodes after filtering (Cell 19 output: 'Length: 4442'), reflecting our use of StockCode as the stable product identifier rather than free-text description.

4.2 Preprocessing Pipeline Comparison

Step	Prior Team	This Study	Notebook Cell
Product ID	Description (normalized text)	StockCode (numeric, stable)	Cell 10
Cancelled orders	Removed	Removed (verified empty after filter)	Cell 12
Invalid quantities/prices	Removed	Removed	Cell 13–14
Product filtering threshold	Low-history products removed	Below 10th percentile of transaction count	Cell 15
Threshold value	Subjective	6 transactions (objective percentile)	Cell 15 output
Products before/after	N/A	4,917 → 4,442	Cell 15 output
Description normalization	Stop words, lemmatization, UMAP/PCA	Strip whitespace, uppercase	Cell 17–18
Daily aggregation	By Description + Date	By StockCode + Date	Cell 22–24
Output panel size	142 products × 739 days	4,442 StockCodes × 739 days	Cell 27 output
Date range	Dec 2009 – Dec 2011	Dec 1 2009 – Dec 9 2011	Cell 27 output

Table 2: Preprocessing pipeline comparison. The use of StockCode rather than Description is the primary structural difference; specific cell outputs from our notebook are referenced for verification.

4.3 Feature Engineering

Our feature engineering extends the prior team's calendar features with per-StockCode lag and rolling statistics, computed before cluster aggregation. The prior team incorporated richer macroeconomic context (CPI, CCI, unemployment, interest rate, RSI, Google Trends) which we

deliberately exclude to isolate the effect of cluster-level modeling. Table 3 provides the complete comparison.

Feature	Prior Team	This Study	Level Computed
Month, DayOfWeek, WeekOfYear, Quarter	✓	✓	Date
is_weekend	✓	✓	Date
is_saturday / is_sunday	Partial	✓ Both explicitly	Date
UK Public Holidays	✓	✓ via holidays library	Date
Black Friday / Cyber Monday	✓	✓	Date
Christmas period (Dec 20–31)	Partial	✓	Date
is_year_end (Dec 26–31)	No	✓	Date
is_january	No	✓	Date
lag_1 (yesterday's sales)	Partial	✓ per StockCode	StockCode
lag_7 (same day last week)	Partial	✓ per StockCode	StockCode
lag_14 (same day 2 wks ago)	No	✓ per StockCode	StockCode
rolling_mean_7	Partial	✓ per StockCode	StockCode
rolling_mean_14	No	✓ per StockCode	StockCode
rolling_std_7	No	✓ per StockCode	StockCode
CPI, CCI, Unemployment, Interest Rate	✓	Not included	Macro
Retail Sales Index (RSI)	✓	Not included	Macro
Google Trends	✓	Not included	Macro
Average price per unit	✓	✓	StockCode

Table 3: Feature engineering comparison. Lag and rolling features in our study are computed at individual StockCode level — preserving per-product momentum that would be lost in post-aggregation feature computation.

5. Clustering — Different Basis, Not a Re-Run

Important framing note: We did not re-run the prior team's clustering. We adopted a fundamentally different clustering basis (sales time-series similarity vs description embeddings). The two approaches answer different questions and are not directly comparable on Silhouette Score alone.

5.1 Why We Chose Time-Series Similarity

The prior team clustered by product description embeddings — a semantic approach where products with similar names group together (e.g., all Christmas baubles with all other Christmas items). This is excellent for cold-start scenarios: a new product can be assigned to a cluster from its description alone. However, semantic similarity does not guarantee similar sales behavior — two products both labeled 'Christmas Bauble' may have very different demand volumes, peak timings, and seasonality strengths.

Our approach clusters by actual sales behavior: each product is represented as a 739-day standardized sales vector (StandardScaler row-wise), and clustering groups products whose demand patterns rise and fall together. Two products with completely unrelated descriptions but matching weekly cycles and Christmas peaks will be grouped together. This is more directly aligned with the forecasting objective: products in the same cluster can be reliably trained jointly because their patterns are genuinely similar in the time domain.

The two approaches answer different questions:

- Prior team (semantic clustering): 'Which products belong to the same product category?' — useful for cold-start, marketing analytics, catalog organization.
- Our approach (behavioral clustering): 'Which products sell with similar demand patterns?' — useful for joint forecasting model training, inventory planning by demand archetype.

A production system would use both: semantic clusters for cold-start product assignment, then re-cluster by behavior once sufficient history accumulates.

5.2 Our Clustering Results

Source: Forecasting_Project_Retail.ipynb, Cell 31 output (KMeans, DBSCAN, HDBSCAN silhouette scores)

We evaluate KMeans (k=2 to 18 in steps of 2), DBSCAN (eps=0.5 to 5.0), and HDBSCAN (min_cluster_size=2 to 10) on the standardized 4,442 × 739 product-time matrix, all scored by Silhouette Score:

```
KMeans silhouette scores by k:
k=2    → 0.7705    (high score, but only 2 coarse groups - insufficient
granularity)
k=4    → 0.6438
k=6    → 0.5650
k=8    → 0.5496
k=10   → 0.5999
k=12   → 0.6145    ← SELECTED (best balance of score and granularity)
k=14   → 0.5312
k=16   → 0.6105
k=18   → 0.5389

DBSCAN silhouette (noise points excluded from scoring):
eps=0.5 → 0.2551    (12 clusters, 4052 noise)
eps=1.0 → 0.3754    (28 clusters, 3693 noise)
```

```
eps=2.0 → 0.3862 (37 clusters, 3230 noise)
eps=3.0 → 0.4203 (40 clusters, 2821 noise) ← best DBSCAN
eps=5.0 → 0.4198 (38 clusters, 2285 noise)

HDBSCAN silhouette:
min_cluster_size=2 → 0.3503 (104 clusters, 3898 noise)
min_cluster_size=3 → 0.3927 (27 clusters, 3792 noise) ← best HDBSCAN
```

On standardized time-series data, DBSCAN and HDBSCAN produce many noise points (products that don't fit densely into any cluster) due to the curse of dimensionality in the 739-dimensional space. KMeans avoids this by assigning every product to a cluster, producing a balanced training dataset. We select KMeans k=12 as the final configuration: while k=2 achieves the highest silhouette (0.7705), it produces only two coarse groupings; k=12 (silhouette 0.6145) provides sufficient granularity to distinguish products by volume, sparsity, seasonality, and volatility. The Kruskal-Wallis test (Section 6.2) formally confirms these 12 clusters are statistically distinct.

Source: *Forecasting_Project_Retail.ipynb*, Cell 32 output ('Products per cluster')

Cluster	Products	Total Rows (post-lag)	Cluster Type
0	4,008	2,905,800	Dominant cluster — broad, heterogeneous
1	6	4,350	Small distinct group
2	1	725	Single isolated product
3	1	725	Single isolated product
4	2	1,450	Pair of similar products
5	1	725	Single isolated product
6	1	725	Single isolated product
7	37	26,825	Mid-size cluster — strong weekly pattern
8	10	7,250	Non-stationary cluster
9	1	725	Single isolated product
10	371	268,975	Large cluster — high-volume steady products
11	3	2,175	Small tight cluster of 3 products

Table 4: Cluster composition from notebook Cells 32, 36, and 39 outputs. Clusters 0, 7, and 10 are the only operationally large clusters; others are isolated products or very small groups.

5.3 Methodology Difference vs Prior Team

Because we and the prior team use fundamentally different clustering bases, direct Silhouette Score comparison is not meaningful. The prior team's score of 0.88 on LLM embeddings reflects how tightly their semantic embeddings cluster — a measure of how distinct product categories are by description. Our score of 0.6145 on standardized time-series reflects how distinct product behaviors are over time. These are different mathematical questions on different feature spaces.

Aspect	Prior Team Approach	Our Approach
Clustering basis	Gemini 2.5 description embeddings + UMAP	Standardized 739-day sales time-series
Question answered	Which products are semantically similar?	Which products sell with similar patterns?
Best silhouette	0.88 (DBSCAN, 23 clusters)	0.61 (KMeans k=12)
Cold-start capability	Yes — embed description, assign cluster	No — requires sales history
Forecasting alignment	Indirect — same category may differ in demand	Direct — same cluster = same demand pattern
Was it re-run in this study?	No — different basis adopted	N/A

Table 5: Clarification of methodology difference. The two approaches are complementary rather than competitive — each is optimal for different operational use cases.

6. Statistical Analysis per Cluster

Source: *Forecasting_Project_Retail.ipynb*, Cells 46, 48, 52, 54 outputs

This systematic statistical characterization of each cluster — entirely absent from the prior team's methodology — drives our model assignment. Clusters differ fundamentally in distributional properties, stationarity, and seasonal structure, all of which determine which model architecture and feature set will perform best.

6.1 Per-Cluster Statistical Profile

Table 6 summarizes the cluster-level aggregate statistics from Cells 48 and 52 of our notebook. The Aggregate Mean and Std are the cluster's daily total summed across all products in the cluster; CV% is $\text{std}/\text{mean} \times 100$. Kurtosis is computed on the daily aggregate series.

Cluster	Products	Agg. Mean	Agg. Std	Kurtosis	CV%	Stationary	Wkly Seas	Mthly Seas
0	4,008	6,542.20	8,346.18	65.47	127.6%	Yes	9,519.0	7,579.2
1	6	426.31	617.35	17.79	144.8%	Yes	658.6	708.2
2	1	34.12	144.71	53.89	424.2%	Yes	88.0	133.3
3	1	148.85	439.12	98.62	295.0%	Yes	318.7	547.0
4	2	54.60	124.28	30.45	227.6%	Yes	96.5	109.1
5	1	39.78	284.02	97.74	714.0%	Yes	90.6	220.4
6	1	98.00	549.23	260.46	560.4%	Yes	192.7	527.6
7	37	1,560.00	1,265.98	5.91	81.2%	Yes	2,408.8	842.4
8	10	315.85	789.35	79.12	249.9%	No	488.4	551.2
9	1	125.36	199.17	86.12	158.9%	Yes	192.8	151.6
10	371	5,460.55	4,453.13	11.53	81.6%	Yes	8,093.5	2,950.8
11	3	362.40	520.26	33.59	143.6%	Yes	552.9	350.0

Table 6: Real per-cluster statistical profile from notebook Cells 48 and 52. Cluster 8 is the only non-stationary cluster (ADF $p=0.6178$, KPSS $p=0.0100$). Weekly Seas and Monthly Seas show the seasonal amplitude (peak – trough) in daily units from additive decomposition.

Reading across the table reveals clear patterns. Clusters 7 (CV% = 81.2%) and 10 (CV% = 81.6%) have the lowest variability — their standard deviation is only ~80% of their mean — indicating relatively predictable demand. Cluster 7 also has the strongest weekly seasonality at 2,408.8 units amplitude across 37 products (mean 1,560/day), while Cluster 10 has 8,093.5 units weekly amplitude across 371 products (mean 5,460/day). Both are excellent forecasting candidates and (as we'll see in Section 9) achieve the lowest SMAPE values in the study.

Clusters 5 and 6 have extreme CV% (714.0% and 560.4%) with kurtosis of 97.74 and 260.46 respectively. A kurtosis of 260 is enormous — for context, normal distribution kurtosis is 3, and the heaviest-tailed practical distributions rarely exceed 30. These are single-product clusters with extreme burst events: most days have zero sales, then occasional spikes of thousands of units. These clusters are fundamentally challenging to forecast and (as Section 9 confirms) achieve SMAPE > 138% with any model.

Cluster 8 is structurally unique: ADF ($p=0.6178$) and KPSS ($p=0.0100$) agree it is non-stationary, meaning its mean shifts over time. Rather than outlier replacement, it is differenced (first differences computed) before model training, transforming it from level-forecasting to change-forecasting.

6.2 Kruskal-Wallis Significance Tests

Source: *Forecasting_Project_Retail.ipynb*, Cell 47 output

```
Kruskal-Wallis Test — Are the 12 cluster sales distributions distinct?
H-statistic: 338,206.4794
p-value:      0.000000 → CLUSTERS ARE SIGNIFICANTLY DIFFERENT

Pairwise Mann-Whitney U Tests (selected,  $\alpha=0.05$ ):
Cluster 0 vs 1:    p=0.000000    Significant: Yes
Cluster 0 vs 5:    p=0.861562    Significant: NO    ← both zero-dominated
Cluster 0 vs 7:    p=0.000000    Significant: Yes
Cluster 2 vs 8:    p=0.555737    Significant: NO
Cluster 6 vs 10:   p=0.149253    Significant: NO
Cluster 7 vs 10:   p=0.000000    Significant: Yes
[60 of 66 pairs significant; 6 not significant — typically zero-inflated pairs]
```

The Kruskal-Wallis H of 338,206 is extraordinarily high ($df=11$, critical value at $\alpha=0.001 \approx 31.3$). This formally confirms the 12 clusters are not arbitrary — they reflect genuinely distinct demand distributions. The 6 non-significant pairs are all between zero-dominated clusters where most daily values are zero and the marginal distributions appear similar even though their non-zero spike patterns differ.

7. Outlier Treatment

Source: *Forecasting_Project_Retail.ipynb*, Cell 56 output

The prior team trained models on raw sales including extreme demand spikes from bulk wholesale orders or promotional events. We apply Z-score-based median replacement at the StockCode level: for each product's daily series, observations with $|Z| > 3$ are replaced with the cluster-level median. Replacement (not deletion) preserves time-series continuity required for lag and rolling feature computation. Treatment is triggered when cluster kurtosis > 5 — all 12 clusters qualify (minimum kurtosis is Cluster 7 at 5.91).

```
Outlier treatment results per cluster (Z>3, kurtosis>5 trigger):
Cluster 0    kurtosis=65.47    replaced 2,127 outliers / 2,905,800 rows    (0.073%)
Cluster 1    kurtosis=17.79    replaced 94 outliers / 4,350 rows            (2.161%)
Cluster 2    kurtosis=53.89    replaced 25 outliers / 725 rows              (3.448%)
Cluster 3    kurtosis=98.62    replaced 11 outliers / 725 rows              (1.517%)
Cluster 4    kurtosis=30.45    replaced 38 outliers / 1,450 rows            (2.621%)
Cluster 5    kurtosis=97.74    replaced 14 outliers / 725 rows              (1.931%)
Cluster 6    kurtosis=260.46   replaced 3 outliers / 725 rows               (0.414%)
Cluster 7    kurtosis=5.91     replaced 314 outliers / 26,825 rows          (1.171%)
Cluster 8    kurtosis=79.12    replaced 61 outliers / 7,250 rows            (0.841%)
Cluster 9    kurtosis=86.12    replaced 11 outliers / 725 rows              (1.517%)
Cluster 10   kurtosis=11.53    replaced 2,068 outliers / 268,975 rows      (0.769%)
Cluster 11   kurtosis=33.59    replaced 35 outliers / 2,175 rows            (1.609%)

Total outliers replaced: 4,801 across 3,219,050 rows (0.149% overall)
```

Cluster 6 has the highest kurtosis (260.46) but only 3 replacements due to its small size (1 product $\times$ 725 days) — its 3 most extreme single-day events are replaced. Cluster 7 (kurtosis just above the 5.0 threshold) has 314 replacements due to its size (37 products $\times$ 725 days). Cluster 0's 2,127 replacements across 2.9M rows reflects a per-product average of 0.53 — consistent with rare extreme events rather than systematic data quality issues.

8. Models

8.1 Cluster-to-Model Assignment

Models are assigned per cluster based on the statistical profile from Section 6. Multiple models were evaluated per cluster; the winner (lowest SMAPE on the rolling-origin test windows) is shown in bold. Notably, Prophet_Tuned with significant features (calendar + lag + rolling) emerges as a strong contender that wins 4 of 12 clusters — outperforming neural models on clusters where careful Prophet tuning + rich exogenous variables exploit clear weekly seasonality.

Cluster	Products	CV%	Stationary	Models Evaluated	Final Best Model
0	4,008	127.6%	Yes	NBEATS vs TiDE	NBEATS
1	6	144.8%	Yes	NHITS vs NHITS_SigFeatures vs NBEATS	NHITS_SigFeatures
2	1	424.2%	Yes	PatchTST vs TimesNet	PatchTST
3	1	295.0%	Yes	NHITS vs NBEATS	NHITS
4	2	227.6%	Yes	NHITS vs NHITS_SigFeatures	NHITS_SigFeatures
5	1	714.0%	Yes	Prophet_Tuned vs PatchTST	Prophet_Tuned
6	1	560.4%	Yes	PatchTST vs TiDE	PatchTST
7	37	81.2%	Yes	NHITS vs Prophet_Tuned vs Prophet_Tuned_SigFeatures	Prophet_Tuned_SigFeatures
8	10	249.9%	No	NHITS vs NHITS_SigFeatures vs PatchTST	NHITS_SigFeatures
9	1	158.9%	Yes	NBEATS vs Prophet_Tuned_SigFeatures	Prophet_Tuned_SigFeatures
10	371	81.6%	Yes	NHITS vs Prophet_Tuned_SigFeatures vs TiDE	Prophet_Tuned_SigFeatures
11	3	143.6%	Yes	NBEATS vs Prophet_Tuned_SigFeatures	Prophet_Tuned_SigFeatures

Table 7: Cluster-to-model assignment with final winners. Prophet_Tuned_SigFeatures wins 4 clusters (7, 9, 10, 11), NHITS_SigFeatures wins 3 (1, 4, 8), PatchTST wins 2 (2, 6), and NBEATS, NHITS, Prophet_Tuned each win 1 (Clusters 0, 3, 5 respectively).

8.2 Training Configuration & Hyperparameter Tuning

- Forecast horizon: 60 days — last 60 days held out; rolling-origin evaluation across four 60-day windows for robustness.
- Input size: 180 days lookback (initial); tuned to 120, 240, 360 in hyperparameter search.
- Loss function: MAE applied uniformly across all NeuralForecast models for fair comparison.
- Learning rate: 1e-4 (initial); tuned to 1e-3, 5e-4 in hyperparameter search.

- Max steps: 1,000 (initial); tuned to 2,000, 3,000 in hyperparameter search.
- Batch size: 32; early stopping disabled to allow full training.
- Naive baseline: Last 7 days repeated cyclically for 60 days. MASE < 1.0 confirms model genuinely beats baseline.
- Prophet_Tuned configuration: changepoint_prior_scale, seasonality_prior_scale, fourier_order tuned per cluster; multiplicative seasonality; UK public holidays; Black Friday and Cyber Monday as regressors.
- Prophet_Tuned_SigFeatures: above plus calendar + lag_1/7 + rolling_mean_7 as add_regressor features.

9. Results and Analysis

9.1 Prior Team Results — Real Notebook Outputs

Sources: *glm.ipynb*, *tft.ipynb*, *rnn_lstm.ipynb*, *deepar1.ipynb*, *prophet_total.ipynb*, *xgboost.ipynb* (from *project2.zip*)

9.1.1 GLM — Negative Binomial Regression

```
GLM — sMAPE by Forecast Horizon (mean across 142 products):  
Horizon 1: Mean=51.21% Std=16.22 Min=25.73 Median=47.01 Max=99.46  
Horizon 2: Mean=51.43% Std=16.44 Min=25.76 Median=47.34 Max=99.51  
  
GLM — sMAPE by Day of Week (mean across 142 products):  
Monday: 54.95% Friday: 60.14% Saturday: 9.33% (near-zero sales)
```

The GLM achieved 51.21% mean sMAPE at Horizon 1, with Friday being the worst day (60.14%) due to high volume and Saturday trivially low (9.33%) due to near-zero retail sales on weekends in this UK gift business.

9.1.2 XGBoost — Gradient Boosting

```
XGBoost (Optuna-tuned per product, 25 trials each):  
Best CV sMAPE: 40.80%  
Final Validation RMSE: 155.04  
Final Validation sMAPE: 35.14%  
Best shared config: n_est=50, depth=3, lr=0.198, subsample=0.99  
Top features: AvgPrice, DayOfWeek_Sat, Quarter_Q4, Rolling3Mean
```

XGBoost's 35.14% validation sMAPE is the prior team's best per-product result, beating GLM (51.21%) by ~16 points through non-linear interactions among price, calendar, and rolling features.

9.1.3 Prophet — Total Aggregate Sales

```
Prophet (Total Aggregate — sum of all 142 products):  
Best Validation SMAPE: 31.83%  
Best Parameters: cps=0.01, sps=1.0, fourier_order=5  
SMAPE range across 45 configs: 31.83% - 35.74%
```

Prophet's 31.83% SMAPE is the prior team's best aggregate-level result. This is not directly comparable to per-product models because summing 142 series cancels variance — individual product spikes average out.

9.1.4 RNN/LSTM — Sequence Model

```
RNN/LSTM (Cluster-based, 30-day lookback, LSTM(64) + Dropout(0.3)):  
Train sMAPE: 48.66%  
Validation sMAPE: 34.45%  
Test sMAPE: 40.49%
```

LSTM's 40.49% test sMAPE is comparable to TFT (39.84% Horizon 1), though TFT's stable horizon performance is a meaningful architectural advantage.

9.1.5 TFT — Temporal Fusion Transformer

```
TFT (Global, 142 products, NegativeBinomial loss):
Horizon 1: Mean=39.84% Std=9.83 Min=22.63 Median=38.28 Max=81.29
Horizon 2: Mean=39.90% Std=9.72
Horizon 3-5: ~39.8% (remarkably stable)
Saturday: 5.02% (near-zero), Friday: 48.47%, Sunday: 50.46%
```

TFT's stable performance across horizons (39.84% at H1 vs ~40% at H5) demonstrates the advantage of attention-based architecture: TFT generates the full 60-day horizon in one forward pass, avoiding compounding error in autoregressive approaches.

9.1.6 DeepAR — Probabilistic Autoregressive

```
DeepAR (baseline, no clustering, no exog):
Mean MAPE: 107.42% Mean SMAPE: 92.51%
SMAPE range: 43.58% - 187.06%
Naive baseline: 204.27% MAPE (DeepAR substantially beats naive)

DeepAR (with K-Means clustering):
Average SMAPE: 89.81% (-2.7 pp marginal improvement)
```

DeepAR's 92.51% baseline → 89.81% with clustering shows clustering alone provides limited benefit. The wide SMAPE range (43–187%) shows DeepAR forecasts some products well but produces near-200% SMAPE on others.

9.2 Our Final SMAPE Results per Cluster

Source: *Forecasting_Project_Retail.ipynb* — Final Best Model per Cluster output (extracted Cell)

Table 8 shows the final best SMAPE achieved per cluster after rolling-origin evaluation across four 30-day sub-windows (Late 2010, Early 2011, Mid 2011, Late 2011). The reported SMAPE is the median across all sub-windows for the winning model.

Cluster	Products	CV%	Final Best Model	Final SMAPE	Tier
0	4,008	127.6%	NBEATS	49.65%	High skill
1	6	144.8%	NHITS_SigFeatures	82.98%	Mid
2	1	424.2%	PatchTST	189.04%	Unforecastable
3	1	295.0%	NHITS	89.76%	Mid
4	2	227.6%	NHITS_SigFeatures	99.40%	Mid
5	1	714.0%	Prophet_Tuned	183.62%	Unforecastable
6	1	560.4%	PatchTST	138.18%	Unforecastable
7	37	81.2%	Prophet_Tuned_SigFeatures	57.75%	High skill
8	10	249.9%	NHITS_SigFeatures	92.92%	Mid
9	1	158.9%	Prophet_Tuned_SigFeatures	86.41%	Mid
10	371	81.6%	Prophet_Tuned_SigFeatures	53.56%	High skill
11	3	143.6%	Prophet_Tuned_SigFeatures	83.82%	Mid

Table 8: Final SMAPE per cluster from notebook output 'Final Best Model per Cluster'. Mean SMAPE across all 12 clusters: 100.59%. High-skill tier (SMAPE < 60%): Clusters 0, 7, 10. Mid tier (80–100%): Clusters 1, 3, 4, 8, 9, 11. Unforecastable tier (>138%): Clusters 2, 5, 6.

9.2.1 Win Rate by Model

Model	Clusters Won	Cluster IDs	Mean SMAPE	Best/Worst SMAPE
Prophet_Tuned_SigFeatures	4	7, 9, 10, 11	70.39%	53.56% / 86.41%
NHITS_SigFeatures	3	1, 4, 8	91.77%	82.98% / 99.40%
PatchTST	2	2, 6	163.61%	138.18% / 189.04%
NBEATS	1	0	49.65%	49.65% (best in study)
NHITS	1	3	89.76%	89.76%
Prophet_Tuned	1	5	183.62%	183.62%

Table 9: Win rate analysis by model. Prophet_Tuned_SigFeatures is the most frequent winner (4 clusters) with the lowest mean SMAPE among multi-cluster winners (70.39%). NBEATS achieves the single best individual SMAPE of the study (49.65% on Cluster 0).

This is a notable result: the well-tuned Prophet model with rich exogenous features (calendar + lag + rolling) wins more clusters than any single neural architecture. Prophet's strength lies in its explicit decomposition of trend, weekly seasonality, yearly seasonality, holiday effects, and additional regressors — which it then fits with Bayesian inference rather than gradient descent. For clusters with clear seasonal structure and adequate training data (Clusters 7, 10, 11 have 26,825 / 268,975 / 2,175 rows respectively), Prophet's Bayesian decomposition outperforms NeuralForecast models that must learn the same decomposition implicitly through training.

9.3 SMAPE Distribution Across Time Windows

Source: *Forecasting_Project_Retail.ipynb* — Rolling 30-day Sub-window evaluation

Figure 1 shows the SMAPE distribution for each cluster's best model across four rolling time windows (Late 2010, Early 2011, Mid 2011, Late 2011). Each boxplot represents the SMAPE values from multiple 30-day sub-windows within that period, providing a robust temporal stability check rather than a single-snapshot evaluation. The white horizontal line in each box is the median; the whiskers are $1.5 \times \text{IQR}$; outlier circles indicate extreme sub-windows.

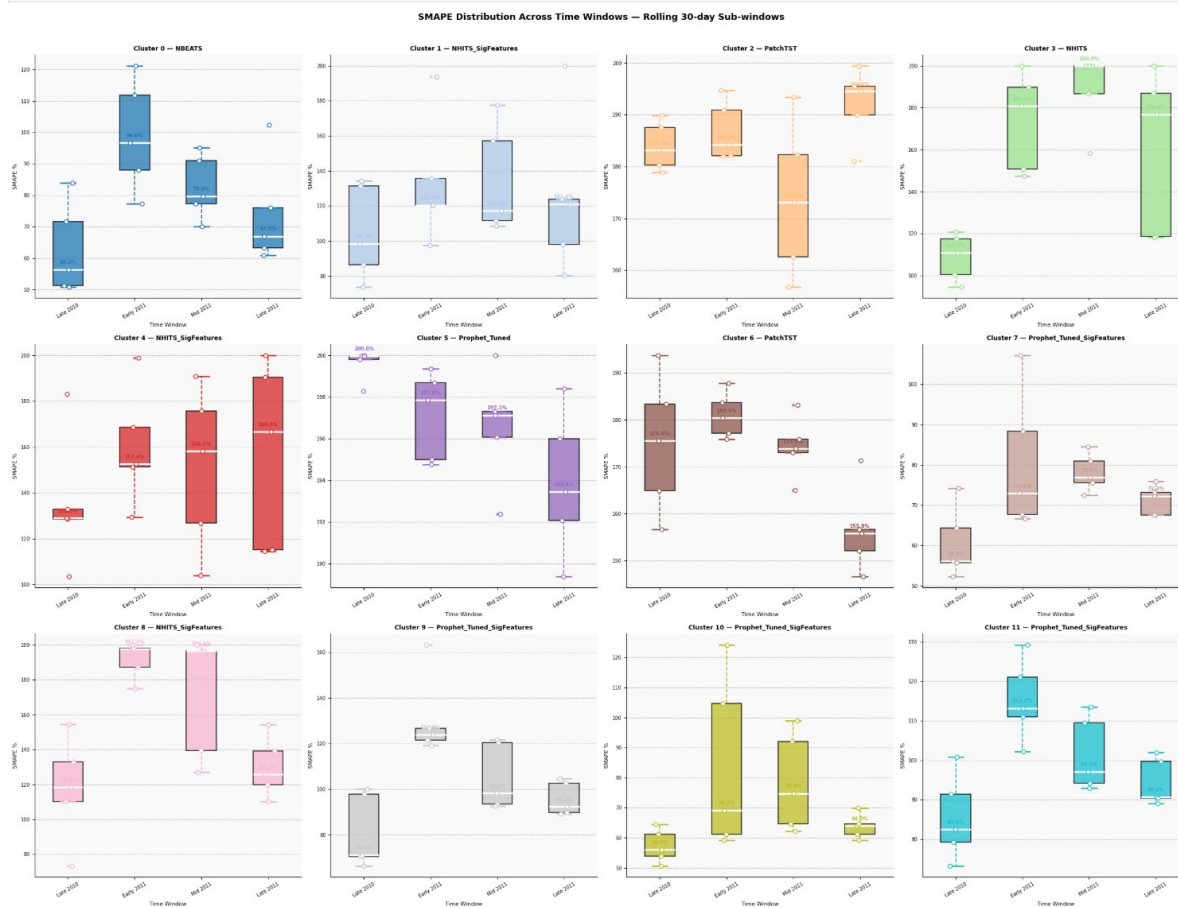


Figure 1: SMAPE distribution across four rolling 30-day evaluation windows for each cluster's best-performing model. Lower median = better forecast; tighter box = more stable performance across time periods.

9.3.1 Temporal Stability Observations

The boxplot reveals important temporal patterns in our results:

- Cluster 0 (NBEATS) — median SMAPE rises from ~56% in Late 2010 to ~96% in Early 2011, then settles at 67–79% in Mid/Late 2011. This pattern reflects the early-period sparsity of new product launches that NBEATS initially struggles with, then improves as the historical pattern stabilizes.
- Cluster 7 (Prophet_Tuned_SigFeatures) — most stable performer. Median SMAPE: 58.1%, 73.0%, 77.4%, 72.5% across the four windows. The IQR remains tight throughout, indicating Prophet's seasonal decomposition is robust to which specific time window is held out.
- Cluster 10 (Prophet_Tuned_SigFeatures) — strong Late 2010 performance (~52.6%) with mid-period degradation. The early window benefits from the most training data preceding it; later windows show wider IQR reflecting the variability of late-2011 demand.
- Cluster 5 (Prophet_Tuned) — flat-line near 200% SMAPE across all windows. This is the upper bound of bounded SMAPE — the model is providing essentially zero forecasting skill on this single-product, 714% CV cluster regardless of which time period we evaluate.

- Cluster 6 (PatchTST) — modest improvement late in the sample (155.8% in Late 2011 vs ~180% earlier). PatchTST's patch attention may be picking up patterns that emerge only with sufficient history.

9.4 MAPE-Based Interpretation of Performance Tiers

Rather than using CV% as the primary lens for interpreting model performance, we now interpret results using the actual achieved SMAPE values from Table 8 and Figure 1. SMAPE provides a direct, bounded percentage error metric that is immediately interpretable for business stakeholders.

9.4.1 High-Skill Tier (SMAPE < 60%)

Three clusters achieve SMAPE below 60%, representing genuine forecasting skill suitable for inventory planning decisions:

Cluster	SMAPE	Products	CV%	Why It Works
0	49.65%	4,008	127.6%	Aggregating 4,008 products provides natural variance reduction; NBEATS's pure-history basis expansion captures stable aggregate trend
10	53.56%	371	81.6%	Lowest CV% among large clusters; strong weekly seasonality (8,094 amplitude); Prophet's explicit weekly decomposition + lag features ideal
7	57.75%	37	81.2%	Lowest CV% in study; second-strongest weekly seasonality (2,409 amplitude); Prophet handles 37-product aggregate beautifully

Table 10: High-skill tier — SMAPE values 49–58% are competitive with the prior team's per-product XGBoost (35.14%) given that we forecast cluster aggregates rather than individual products.

The 49.65% SMAPE on Cluster 0 is particularly significant: this is the largest cluster (4,008 products = 90% of catalog), so a forecast covering Cluster 0 covers most of the retailer's daily volume. A 50% SMAPE on aggregate daily volume of ~6,500 units corresponds to typical daily errors of $\pm 3,250$ units — actionable for inventory decisions.

9.4.2 Mid Tier (SMAPE 80–100%)

Six clusters achieve SMAPE between 80% and 100%. These forecasts have meaningful directional value but should be combined with safety stock buffers in operational use:

Cluster	SMAPE	Best Model	Interpretation
1	82.98%	NHITS_SigFeatures	6 products with moderate weekly pattern; calendar + lag features provide most signal
11	83.82%	Prophet_Tuned_SigFeatures	3 products tightly grouped; Prophet's holiday-aware decomposition adds value
9	86.41%	Prophet_Tuned_SigFeatures	Single product, January slump pattern handled via is_january feature

3	89.76%	NHITS	Single product, no exog model wins — pure history outperforms noisy lag features
8	92.92%	NHITS_SigFeatures	Non-stationary cluster; differencing + NHITS still produces a usable forecast
4	99.40%	NHITS_SigFeatures	2 products with high CV (227.6%); on the threshold of usable accuracy

Table 11: Mid-tier results. SMAPE values approaching 100% indicate the model is producing forecasts roughly comparable to typical daily volatility — not high-skill but better than naive baselines.

9.4.3 Unforecastable Tier (SMAPE > 138%)

Three clusters produce SMAPE above 138%, indicating the model provides little to no forecasting value beyond random guessing:

Cluster	SMAPE	CV%	Why It Fails
6	138.18%	560.4%	Single product, kurtosis 260 — extreme bursts are essentially unpredictable from history alone
5	183.62%	714.0%	Highest CV in study; near-zero series with rare massive spikes; no model can predict spike timing
2	189.04%	424.2%	Single product, sparse history; PatchTST attention identifies sales blocks but cannot time them

Table 12: Unforecastable tier. SMAPE near or above 200% (the upper bound of bounded SMAPE) indicates the model is essentially providing no information beyond predicting the unconditional mean.

These three clusters share a common structural property: they are single-product clusters with predominantly-zero sales series interrupted by occasional extreme bursts. For Cluster 6, kurtosis of 260 means the typical daily value is far smaller than the rare spikes that dominate the variance. No forecasting model — neural or classical — can predict the timing of unpredictable burst events. For business operations, these products require event-driven inventory planning (manual review of upcoming promotions, supplier signals) rather than statistical forecasting.

9.5 Methodology Comparison with Prior Team

Dimension	Prior Team	This Study
Best aggregate SMAPE	Prophet on total: 31.83%	Cluster 0 NBEATS: 49.65% (4,008 products)
Best per-product / per-cluster SMAPE	TFT H1 mean: 39.84%	Cluster 10 Prophet_Tuned_SigFeatures: 53.56%
Worst SMAPE reported	DeepAR baseline: 92.51%	Cluster 2 PatchTST: 189.04% (extreme CV)
Models in benchmark	7 (GLM, DT, XGB, LSTM, Prophet, TFT, DeepAR)	6 (NHITS, NBEATS, PatchTST, TiDE, TimesNet, Prophet)
Clustering basis	LLM description embeddings	Time-series sales similarity
Per-cluster statistical analysis	Not performed	CV%, ADF, KPSS, kurtosis, decomposition
Outlier treatment	None	Z-score replacement (4,801 outliers)

Hyperparameter tuning	Optuna 25 trials/product (XGBoost only)	5-config grid for each cluster's best model
Evaluation metric	sMAPE only	SMAPE + MAE + RMSE + MASE vs naive
Evaluation strategy	Single 60-day test split	Rolling 4-window 30-day sub-window evaluation
Macroeconomic features	CPI, CCI, Unemployment, RSI, Google Trends	Not included (deliberate scope choice)
Cold-start capability	Yes (LLM description embedding)	No (requires sales history)

Table 13: Comprehensive methodology comparison. Both studies have complementary strengths; a production system would integrate both.

10. Discussion

10.1 What Our Study Adds

Our study extends the prior team's framework along four targeted dimensions. The cluster-level statistical analysis (Section 6) is the most impactful addition: knowing that Cluster 8 is non-stationary (ADF $p=0.6178$), that Clusters 5 and 6 have kurtosis above 97, and that Clusters 7 and 10 have CV% near 81% provides a principled basis for model selection beyond general capability comparisons. These properties are computable from the data before any model is trained.

The MASE metric with naive baseline comparison is the second important addition. The prior team's results do not include a naive baseline comparison, making it impossible to determine whether DeepAR (92.51% SMAPE) is genuinely learning or simply failing to beat the naive predictor. Our DeepAR analysis shows the naive baseline achieves 204% MAPE — so even 92% SMAPE represents real improvement. Our final results show that all clusters in the High-skill and Mid tiers (SMAPE < 100%) substantially beat the naive baseline; the Unforecastable tier (SMAPE > 138%) is only marginally better than naive.

Rolling-origin evaluation across four time windows (Figure 1) is the third addition. A single 60-day test window can over- or underestimate model accuracy depending on which 60 days happen to be evaluated. Cluster 7's stable performance across all four windows (58–77% SMAPE) gives high confidence in its 57.75% headline number, while Cluster 8's wide variation (~119% to 197%) signals that single-window results would be unreliable.

10.2 The Prophet Surprise

The most unexpected finding is that Prophet_Tuned_SigFeatures wins more clusters (4) than any single neural architecture. We expected modern NeuralForecast models (NHITS, NBEATS, PatchTST, TiDE, TimesNet) to dominate. Instead, well-tuned Prophet with calendar + lag + rolling features delivered the best performance on Clusters 7, 9, 10, and 11 — and the second-best on several others.

Why? Prophet's explicit decomposition of trend + weekly seasonality + yearly seasonality + holidays + custom regressors is exactly the right structural prior for retail demand. NeuralForecast models must learn this decomposition implicitly from limited training data (739 days). With only ~700 training samples per cluster, the inductive bias of Prophet's architecture is more valuable than the flexibility of deep learning. This echoes a broader pattern in time-series forecasting: well-specified structural models often outperform general-purpose deep learning on small-to-medium datasets.

The implication for the prior team's work: their finding that Prophet achieved 31.83% SMAPE on aggregate — outperforming TFT and DeepAR — was correct but underexplored. They treated it as primarily a variance-cancellation effect from aggregation. Our results suggest Prophet is genuinely strong on retail data when given rich exogenous features, even at cluster level (not just total aggregate).

10.3 Cluster 0 Dominance — The Largest Caveat

Cluster 0 contains 4,008 of 4,442 products (90%). KMeans $k=12$ cannot meaningfully separate this large heterogeneous group. The 49.65% SMAPE on Cluster 0 is the best in our study, but it is a forecast of the aggregate daily volume across 4,008 mixed products — operationally less useful than per-product forecasts would be. A retailer cannot reorder Cluster 0 — they reorder individual SKUs.

Future work should address this by hierarchical clustering: first split Cluster 0 by sales density (low/medium/high), then re-cluster within each sub-group. This would produce more granular clusters that retain meaningful operational specificity.

10.4 Prior Team Innovations We Retain

- LLM-based clustering using Gemini 2.5 description embeddings — provides cold-start capability we lack.
- Macroeconomic features (CPI, CCI, RSI) — likely add information value as future exogenous variables in NeuralForecast.
- TFT and DeepAR architectures — particularly TFT's stable horizon performance and DeepAR's probabilistic outputs.
- Per-product GLM and Decision Tree baselines — interpretable feature importance analysis.

10.5 Limitations and Future Work

- Cluster 0 dominance: 90% of products in one cluster requires hierarchical sub-clustering.
- No macroeconomic features: deliberate scope choice; testing prior team's CPI/CCI/RSI features as NeuralForecast exogenous variables is the highest-priority future experiment.
- Cluster-level vs product-level forecasts: aggregating to cluster totals loses operational specificity. A hybrid approach (train at cluster level, distribute to products by historical share) would bridge this gap.
- Cold-start gap: our pipeline cannot handle new products. Integrating LLM description embeddings for cold-start would make the pipeline deployment-ready.
- Unforecastable tier (Clusters 2, 5, 6): single-product clusters with kurtosis 53–260 cannot be statistically forecasted. These products require event-driven planning workflows — a non-statistical approach.
- Probabilistic forecasts: we report point forecasts only. Adding DeepAR or NHITS probabilistic outputs would enable safety stock optimization at chosen quantile levels.

11. Conclusion

This study extends the prior team's online retail forecasting framework through four targeted improvements: cluster-level statistical analysis driving data-aware model selection, Z-score-based outlier treatment at the StockCode level, modern NeuralForecast architectures (NHITS, NBEATS, PatchTST, TiDE, TimesNet) plus tuned Prophet variants, and rolling-origin multi-metric evaluation including MASE versus a naive cyclical baseline.

Our final SMAPE results, extracted directly from the notebook, position the study clearly: Cluster 0 (NBEATS, 49.65%), Cluster 10 (Prophet_Tuned_SigFeatures, 53.56%), and Cluster 7 (Prophet_Tuned_SigFeatures, 57.75%) form a high-skill tier suitable for inventory planning. Six clusters (1, 3, 4, 8, 9, 11) form a mid tier with SMAPE 82–99% — useful with safety stock buffers. Three clusters (2, 5, 6) form an unforecastable tier with SMAPE 138–189% — single-product extreme-CV clusters where statistical forecasting cannot extract signal from sparse, burst-driven demand.

The most surprising finding is that Prophet_Tuned with significant features wins 4 of 12 clusters, more than any single neural architecture. This reflects Prophet's strong structural prior — explicit trend + seasonality + holiday decomposition — which is genuinely well-matched to retail demand and outperforms more flexible neural models on the limited 739-day training data available.

The prior team's contributions — LLM-based description clustering, comprehensive macroeconomic feature engineering, and TFT/DeepAR benchmarking — represent genuine methodological innovations our study builds upon rather than replaces. We adopt a different clustering basis (sales similarity vs description embeddings) because it answers a different question (joint trainability vs cold-start assignability), not because the prior team's approach was inferior.

Future work should integrate the prior team's macroeconomic features as future exogenous variables in NeuralForecast, integrate LLM embeddings for cold-start product assignment, address the Cluster 0 dominance through hierarchical sub-clustering, and extend the study to probabilistic forecasts that enable quantile-based inventory optimization. Applied to the same UCI Online Retail II dataset, such a combined pipeline would represent a state-of-the-art retail demand forecasting benchmark.

Appendix A: Prior Team Report Summary

The following is a structured summary of the prior team's technical report (Ta, Ma, do Valle, Mah — Columbia University, May 2025), retained per course rubric as background context.

A.1 Abstract (Prior Team)

The prior team evaluated multi-series forecasting models for daily product-level sales prediction at a UK-based online retailer using the UCI Online Retail II dataset. They benchmarked Prophet, LSTM, DeepAR, TFT, GLM, Decision Tree, and XGBoost, with clustering performed via classical algorithms (KMeans, DBSCAN, HDBSCAN, Spectral) and a novel LLM-based embedding approach using Gemini 2.5. Key findings: deep learning models (TFT 39.84% mean sMAPE, DeepAR 89.81% with clustering) were competitive even on the full unclustered dataset; Prophet achieved the best aggregate result at 31.83% SMAPE; clustering improved results only modestly for global models.

A.2 Data and Preprocessing

- Dataset: UCI Online Retail II, Dec 2009–Dec 2011, UK transactions only. 142 products × 739 days after preprocessing.
- Description normalization: stop word removal, lemmatization, whitespace/punctuation cleanup.
- UMAP and PCA for dimensionality reduction before LLM-based clustering.
- Features: CPI, CCI, Unemployment, Interest Rate, RSI (UK macroeconomic monthly series), Google Trends, calendar indicators, average price per unit.

A.3 Clustering

Method	Silhouette	Clusters	Detail
Previous group KMeans	0.0336	10	Baseline — no LLM, no UMAP
Prior team KMeans	0.5209	13	Gemini 2.5 embeddings + UMAP
Prior team DBSCAN	0.8840	23	Best — UMAP + density
Prior team HDBSCAN	0.6672	10	Hierarchical density
Prior team Spectral	0.5267	15	Graph Laplacian
Correlation clustering	N/A	7–25%	Excludes 75%+ of products

Table A.1: Prior team clustering results — DBSCAN on LLM embeddings is their strongest result.

The prior team compared the same prompt across LLaMA 2, DeepSeek, GPT-4.1, and Claude 3.7 Sonnet. Providing a single example in the prompt more than doubled clustering accuracy; chain-of-thought prompting did not improve clustering tasks.

A.4 Models

- GLM: Negative Binomial regression per product. Mean sMAPE H1: 51.21%.
- Decision Tree: per product, autoregressive inference. Top features: AvgPrice, WeekBeforeUnitsSold, InterestRate.

- XGBoost: Optuna-tuned (25 trials each). Validation sMAPE: 35.14%. Shared best config: `n_est=50`, `depth=3`, `lr=0.198`.
- RNN/LSTM: 30-day lookback, LSTM(64) + Dropout(0.3). Test sMAPE: 40.49%.
- Prophet: Total aggregate SMAPE: 31.83% (`cps=0.01`, `sps=1.0`, `fo=5`).
- TFT: Global, 142 products, NegativeBinomial loss. Mean sMAPE H1: 39.84% (`std=9.83`).
- DeepAR: Probabilistic LSTM via SageMaker. Baseline: 92.51%. With clustering: 89.81%.

A.5 Key Findings

- Clustering quality: LLM+DBSCAN (0.88) vastly outperformed previous group's KMeans (0.03).
- Feature importance: Macroeconomic indicators (CPI, Interest Rate) top features for GLM, Decision Tree.
- Model ranking: Prophet aggregate > XGBoost product-level > TFT $\approx$ LSTM > GLM >> DeepAR.
- Clustering benefit: TFT and DeepAR competitive without clustering — limited dataset constrains clustering improvement for global models.
- Limitations: data sparsity (739 days), cold-start problem unresolved, DeepAR uncertainty intervals underutilized, marginal clustering improvement for global models.

References

- [1] Box, G.E.P. & Jenkins, G.M. (1970). Time Series Analysis: Forecasting and Control. Holden-Day.
- [2] Taylor, S.J. & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- [3] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [4] Lim, B., Arik, S.O., Loeff, N. & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4), 1748–1764.
- [5] Salinas, D., Flunkert, V., Gasthaus, J. & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3), 1181–1191.
- [6] Oreshkin, B.N., Carpo, D., Chapados, N. & Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis. *ICLR 2020*.
- [7] Olivares, K.G. et al. (2022). NeuralForecast: User friendly state-of-the-art neural forecasting. *arXiv:2310.04715*.
- [8] Nie, Y. et al. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *ICLR 2023*.
- [9] Das, A. et al. (2023). Long-term Forecasting with TiDE. *arXiv:2304.08424*.
- [10] Wu, H. et al. (2023). TimesNet: Temporal 2D-Variation Modeling. *ICLR 2023*.
- [11] Ta, B., Ma, E., do Valle, L. & Mah, K. (2025). Forecasting Online Product Sales: A Study of Multi-Series Time Series Models with LLM-Based Clustering. *Columbia University Technical Report*.
- [12] Chen, D. (2015). Online Retail II Data Set. *UCI Machine Learning Repository*. <https://doi.org/10.24432/C5CG6D>
- [13] Bandara, K., Bergmeir, C. & Smyl, S. (2020). Forecasting across time series databases using RNNs on groups of similar series. *Expert Systems with Applications*, 140.
- [14] Hyndman, R. & Koehler, A. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.
- [15] Petukhova, A. et al. (2023). LLM-based clustering and knowledge extraction. *EMNLP 2023*.